

# LAB 04.2: Mapping Process Runtime Topography Using Low-Level Hardware Debugging Infrastructure


**Aaisma Wagle**

**Bsc (Hons) Computer Science with Artificial Intelligence**

Softwarica College of IT and E-commerce


***ST5003CMD Operating Systems, Security, and Networks***


**Rupak Rajbanshi**


10th October, 2026

# TABLE OF CONTENT

**TABLE OF CONTENT** **2**

**LIST OF FIGURES** **3**

**INTRODUCTION** **1**

**TARGET LABORATORY SOURCE CODE** **2**

Task A: Building with Debugging Symbols 3

Task B: Setting Boundaries and Runtime Positioning 3

Task C: Mapping Static BSS Segment Bounds 4

Task D: Extracting OS Kernel Address Tables 7

Task E: Auditing Local Register States and Low-Level Cells 8

**POST-LAB TECHNICAL ASSESSMENT** **9**

**CONCLUSION** **10**

**GITHUB REPOSITORY** **10**

**REFERENCES** **10**

# LIST OF FIGURES

**Figure 01:** demo.c File in Nano Editor **2**

**Figure 02:** demo.c File View using cat Command **2**

**Figure 03:** Compile demo.c using the -g optimization preservation flag **3**

**Figure 04:** demo.c File in Nano Editor **3**

**Figure 05:** Section Ranges from info files **4**

**Figure 06:** Program Headers (readelf -l) **5**

**Figure 07:** Section to Segment Mapping (readelf -l) **5**

**Figure 08:** Address of bss_global_var at the Breakpoint **6**

**Figure 09:** Live Memory Map from info proc mappings **7**

**Figure 10:** Stack Variable and Stack Pointer Contents **8**

**Figure 11:** Stack Variable and Stack Pointer Contents **8**

**Figure 12:** Program Finished and GDB Closed **9**

# LIST OF TABLES

**Table I:** Addresses Printed by demo Inside GDB **3**

**Table II:** Details of the RW LOAD Segment (Segment 05) from readelf -l **5**

**Table III:** Static Address vs Runtime Address of bss_global_var **6**

**Table IV:** Start and Stop Virtual Addresses of .bss, [heap] and [stack] **7**

# INTRODUCTION

This is the lab report to map how a running process is laid out in virtual memory. When a program runs, the operating system gives it its own private virtual address space instead of direct access to physical RAM. This space is divided into regions. Each region stores a different kind of data. Global variables that are not initialized go in the .bss segment, local variables live on the stack, and memory requested at runtime with malloc() comes from the heap (Bryant & O'Hallaron, 2015).

This lab is done to answer where these regions sit in the address space, how we can prove that a variable belongs to a certain region, and how the stack and the heap are placed relative to each other. To do this, the GNU Debugger (GDB) is used to pause a program while it is running and inspect its memory. readelf is used to compare the static layout of the file on disk with the live layout shown by the kernel (Kerrisk, 2010).

**Sudo mkdir <repository name>** //For creating a new folder.
(Sudo is used to access files that regular user account doesn't have access to)

**ls** //list all files in the directory

**sudo nano <file name>** //Opens the nano text editor to write or edit a C file

**cat <file name>** //Shows the contents of a file in the terminal

**cd <folder name>** //Moves into a folder

# TARGET LABORATORY SOURCE CODE

A new file called ‘demo.c’ was made. This C program creates one variable in each major memory region so we can find them later with GDB. **bss_global_var** is a global variable with no starting value. It is placed in the **.bss** segment and is set to 0 by default. **stack_local_var** is a local variable inside main(). It is stored on the stack and holds the value 42. **heap_ptr** points to memory requested with malloc(). That memory comes from the heap and holds the value 99.

The program also prints the address of each one using **printf()** with the %p format.

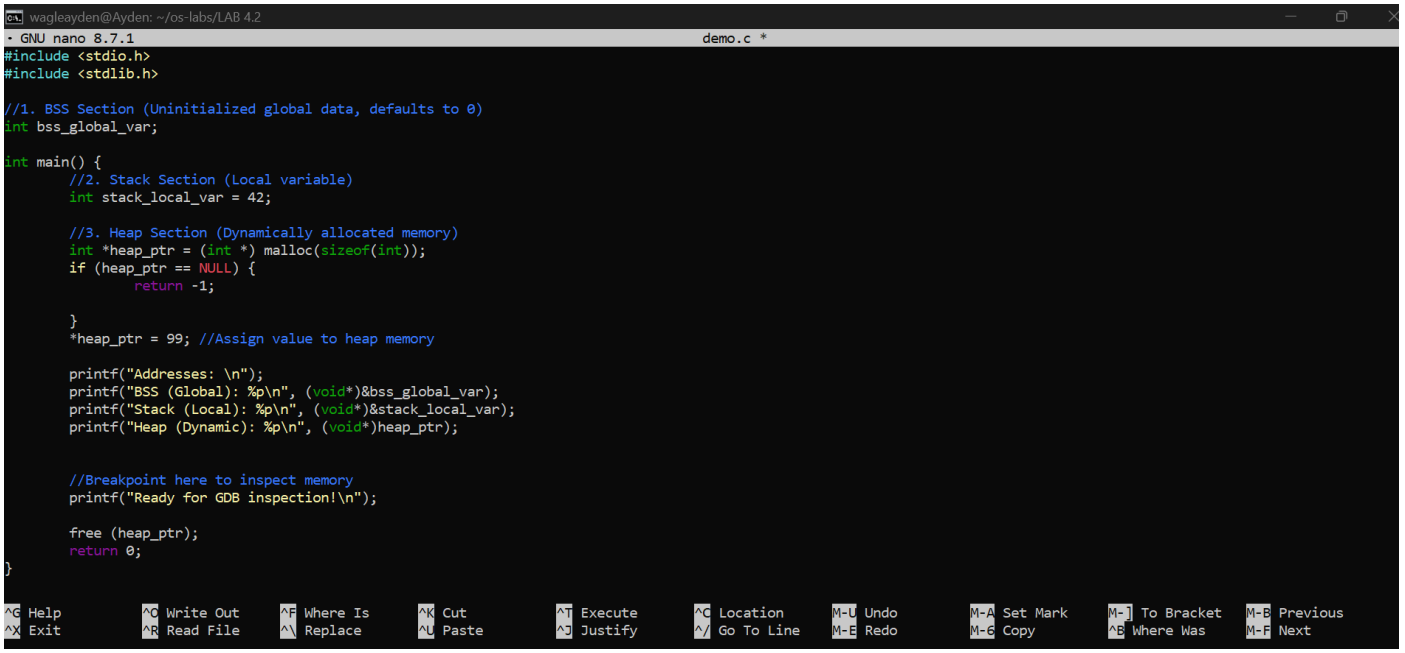



*Figure 01: demo.c File in Nano Editor*

```
wagleayden@Ayden:~/os-labs/LAB 4.2$ sudo touch demo.c
wagleayden@Ayden:~/os-labs/LAB 4.2$ ls
demo.c
wagleayden@Ayden:~/os-labs/LAB 4.2$ sudo nano demo.c
wagleayden@Ayden:~/os-labs/LAB 4.2$ cat demo.c
#include <stdio.h>
#include <stdlib.h>

//1. BSS Section (Uninitialized global data, defaults to 0)
int bss_global_var;

int main() {
        //2. Stack Section (Local variable)
        int stack_local_var = 42;

        //3. Heap Section (Dynamically allocated memory)
        int *heap_ptr = (int *) malloc(sizeof(int));
        if (heap_ptr == NULL) {
                return -1;

        }
        *heap_ptr = 99; //Assign value to heap memory

        printf("Addresses: \n");
        printf("BSS (Global): %p\n", (void*)&bss_global_var);
        printf("Stack (Local): %p\n", (void*)&stack_local_var);
        printf("Heap (Dynamic): %p\n", (void*)heap_ptr);


        //Breakpoint here to inspect memory
        printf("Ready for GDB inspection!\n");

        free (heap_ptr);
        return 0;
}
```

*Figure 02: demo.c File View using cat Command*

## Task A: Building with Debugging Symbols

**gcc -g <source file> -o <executable file>** //To compile the program with debugging symbols (-g keeps variable names and line numbers in the binary).

```
wagleayden@Ayden:~/os-labs/LAB 4.2$ sudo gcc -g demo.c -o demo
wagleayden@Ayden:~/os-labs/LAB 4.2$ ls
demo  demo.c
```

*Figure 03: Compile demo.c using the -g optimization preservation flag*

## Task B: Setting Boundaries and Runtime Positioning

**gdb ./<executable file>** //To open the program inside the debugger

**break 23** //To pause the program at line 23

**run** //To start the program

```
wagleayden@Ayden:~/os-labs/LAB 4.2$ gdb ./demo
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./demo...
(gdb) break 23
Breakpoint 1 at 0x1277: file demo.c, line 26.
(gdb) run
Starting program: /home/wagleayden/os-labs/LAB 4.2/demo
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Addresses:
BSS (Global): 0x555555558014
Stack (Local): 0x7fffffffde6c
Heap (Dynamic): 0x555555559010

Breakpoint 1, main () at demo.c:26
26              printf("Ready for GDB inspection!\n");
(gdb)
```

*Figure 04: demo.c File in Nano Editor*

I typed break 23, but GDB reported line 26. This is because line 23 has no executable code so GDB moves the breakpoint to the next line that does. That line is the **printf("Ready for GDB inspection!")** call, so the pause happens at the right place, after malloc() and before free().

The program printed these addresses while running under GDB:

*Table I: Addresses Printed by demo Inside GDB*

| Variable | Region | Address |
|---|---|---|
| bss_global_var | BSS | 0x555555558014 |
| stack_local_var | Stack | 0x7fffffffde6c |

| heap_ptr | Heap | 0x555555559010 |
|---|---|---|

The BSS and heap addresses start with 0x5555..., while the stack address starts with 0x7fff.... This already shows that the stack sits much higher in memory than the other two regions.

## Task C: Mapping Static BSS Segment Bounds

**info files** //Shows the memory range of every section of the running program

```
(gdb) info files
Symbols from "/home/wagleayden/os-labs/LAB 4.2/demo".
Native process:
        Using the running image of child process 484.
        While running this, GDB does not access memory from...
Local exec file:
        `/home/wagleayden/os-labs/LAB 4.2/demo', file type elf64-x86-64.
        Entry point: 0x5555555550e0
        0x0000555555554350 - 0x0000555555554374 is .note.gnu.build-id
        0x0000555555554374 - 0x0000555555554390 is .interp
        0x0000555555554390 - 0x00005555555543b4 is .gnu.hash
        0x00005555555543b8 - 0x00005555555544c0 is .dynsym
        0x00005555555544c0 - 0x000055555555457b is .dynstr
        0x000055555555457c - 0x0000555555554592 is .gnu.version
        0x0000555555554598 - 0x00005555555545d8 is .gnu.version_r
        0x00005555555545d8 - 0x0000555555554698 is .rela.dyn
        0x0000555555554698 - 0x0000555555554710 is .rela.plt
        0x0000555555555000 - 0x000055555555501b is .init
        0x0000555555555020 - 0x0000555555555080 is .plt
        0x0000555555555080 - 0x0000555555555090 is .plt.got
        0x0000555555555090 - 0x00005555555550e0 is .plt.sec
        0x00005555555550e0 - 0x00005555555552ad is .text
        0x00005555555552b0 - 0x00005555555552bd is .fini
        0x0000555555556000 - 0x0000555555556063 is .rodata
        0x0000555555556064 - 0x0000555555556098 is .eh_frame_hdr
        0x0000555555556098 - 0x0000555555556144 is .eh_frame
        0x0000555555556148 - 0x0000555555556178 is .note.gnu.property
        0x0000555555556178 - 0x0000555555556198 is .note.ABI-tag
        0x0000555555557d98 - 0x0000555555557da0 is .init_array
        0x0000555555557da0 - 0x0000555555557da8 is .fini_array
        0x0000555555557da8 - 0x0000555555557f98 is .dynamic
        0x0000555555557f98 - 0x0000555555558000 is .got
        0x0000555555558000 - 0x0000555555558010 is .data
        0x0000555555558010 - 0x0000555555558018 is .bss
        0x00007ffff7fc02a8 - 0x00007ffff7fc02cc is .note.gnu.build-id in /lib64/ld-linux-x86-64.so.2
        0x00007ffff7fc02d0 - 0x00007ffff7fc0408 is .hash in /lib64/ld-linux-x86-64.so.2
        0x00007ffff7fc0408 - 0x00007ffff7fc0564 is .gnu.hash in /lib64/ld-linux-x86-64.so.2
        0x00007ffff7fc0568 - 0x00007ffff7fc0910 is .dynsym in /lib64/ld-linux-x86-64.so.2
        0x00007ffff7fc0910 - 0x00007ffff7fc0bc1 is .dynstr in /lib64/ld-linux-x86-64.so.2
        0x00007ffff7fc0bc2 - 0x00007ffff7fc0c10 is .gnu.version in /lib64/ld-linux-x86-64.so.2
```

*Figure 05: Section Ranges from info files*

The .bss row of the program is from 0x0000555555558010 - 0x0000555555558018. The .data section ends at 0x555555558010, exactly where .bss starts. Initialized globals go in .data and uninitialized ones go in .bss, so the two sit side by side. **&bss_global_var** shows 0x555555558014 so it is inside the given range.

### readelf -l: to read program header

**readelf -l <executable file>** //Reads the program headers of the ELF file

```
wagleayden@Ayden:~/os-labs/LAB 4.2$ readelf -l demo

Elf file type is DYN (Position-Independent Executable file)
Entry point 0x10e0
There are 14 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz            MemSiz              Flags  Align
  PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
                 0x0000000000000310 0x0000000000000310  R      0x8
  INTERP         0x0000000000000374 0x0000000000000374 0x0000000000000374
                 0x000000000000001c 0x000000000000001c  R      0x1
      [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
  LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x0000000000000710 0x0000000000000710  R      0x1000
  LOAD           0x0000000000001000 0x0000000000001000 0x0000000000001000
                 0x00000000000002bd 0x00000000000002bd  R E    0x1000
  LOAD           0x0000000000002000 0x0000000000002000 0x0000000000002000
                 0x0000000000000198 0x0000000000000198  R      0x1000
  LOAD           0x0000000000002d98 0x0000000000003d98 0x0000000000003d98
```

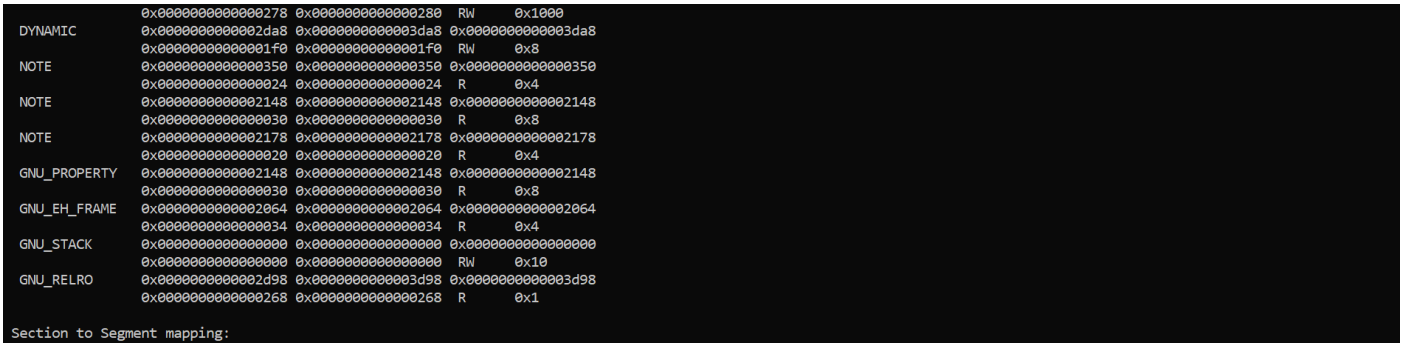



*Figure 06: Program Headers (readelf -l)*




*Figure 07: Section to Segment Mapping (readelf -l)*

The .bss section is in segment 05, together with .init_array, .fini_array, .dynamic, .got and .data.

*Table II: Details of the RW LOAD Segment (Segment 05) from readelf -l*

| **Field** | **Value** |
|---|---|
| Type | LOAD |
| VirtAddr | 0x3d98 |
| FileSiz | 0x278 |
| MemSiz | 0x280 |
| Flags | RW |

FileSiz is the size stored in the file, and MemSiz is the size when the program is loaded into memory. MemSiz is 8 bytes bigger $0x280 - 0x278 = 0x8$. This extra 8 bytes is the .bss, which is all zeros, so it is not stored in the file but the loader still reserves memory for it.

**print &<variable name>** //Shows the memory address of a variable




*Figure 08: Address of bss_global_var at the Breakpoint*

While the program was paused at the breakpoint, **print &bss_global_var** returned:

$1 = (int *) 0x555555558014 <bss_global_var>

This matches the address that the program printed itself with printf() so GDB and the running program agree. It also falls inside the .bss range from info files which confirms that the global variable is stored in the .bss segment. The address was also checked before the program was started. At that point GDB showed only the file offset, because the program had not been loaded into memory yet.

*Table III: Static Address vs Runtime Address of bss_global_var*

| Moment | Address of bss_global_var |
|---|---|
| Before run (file layout) | 0x4014 |
| At the breakpoint (in memory) | 0x555555558014 |

The two addresses are different because the program is a PIE (Position-Independent Executable). In the file on disk, addresses start from 0. When the program runs, the operating system loads it into memory at a starting address called the base address.

Here the base address is 0x555555554000. The real address in memory is the file address plus the base address. $0x555555554000 + 0x4014 = 0x555555558014$

The same works for the end of .bss. In the file it ends at 0x4018.

$0x555555554000 + 0x4018 = 0x555555558018$. This is the same end address that info files showed, so readelf and GDB agree.

## Task D: Extracting OS Kernel Address Tables

**info proc mappings** //Shows the live virtual memory map of the running process

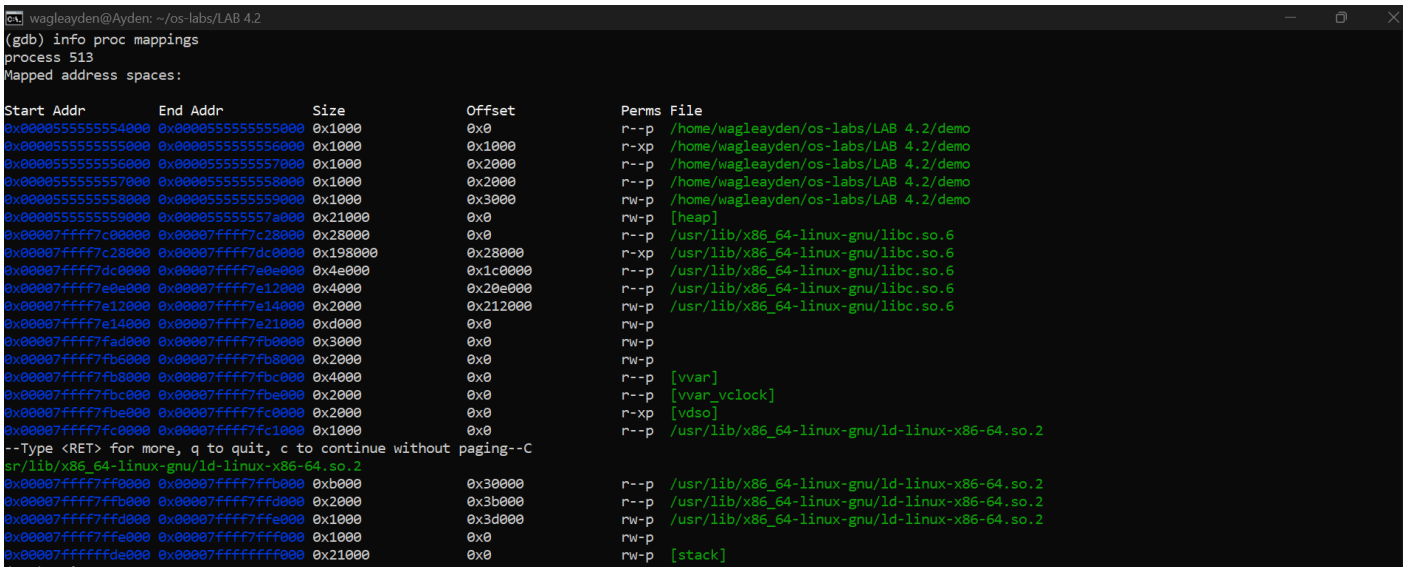



*Figure 09: Live Memory Map from info proc mappings*

The kernel's map lists every region of the process with its start address, end address, size and permissions. The [heap] and [stack] rows were:

*Table IV: Start and Stop Virtual Addresses of .bss, [heap] and [stack]*

| Region | Start Address | Stop Address |
|---|---|---|
| .bss | 0x555555558010 | 0x555555558018 |
| [heap] | 0x555555559000 | 0x55555557a000 |
| [stack] | 0x7fffffde000 | 0x7fffffff000 |

The .bss row comes from info files. The heap and stack rows come from info proc mappings.

Checking the variables against the table, it can be seen:

**heap_ptr = 0x555555559010**. This is between 0x555555559000 and 0x55555557a000, so it is inside **[heap]**.

**stack_local_var = 0x7fffffffde6c**. This is between 0x7fffffde000 and 0x7fffffff000, so it is inside **[stack]**.

The .bss and heap are both near 0x5555..., right after the program's own mapping. The stack is far above them near 0x7fff.... Both heap and stack are currently 0x21000 bytes or 132 KB.

## Task E: Auditing Local Register States and Low-Level Cells

**print <variable name>** //Shows the value of a variable

**print &<variable name>** //Shows the address of a variable

**x/4wx $sp** //Shows 4 words of raw memory from the stack pointer, in hex

```
(gdb) print stack_local_var
$2 = 42
(gdb) print &stack_local_var
$3 = (int *) 0x7fffffffde6c
(gdb) x/4wx $sp
0x7fffffffde60: 0x00000000      0x00000000      0x00000000      0x0000002a
```

*Figure 10: Stack Variable and Stack Pointer Contents*

**print stack_local_var** gave 42, the value set in the code.

**print &stack_local_var** gave 0x7fffffffde6c which is the same address the program printed.

**x/4wx $sp** started at 0x7fffffffde60 and showed 0x00000000 0x00000000 0x00000000 0x0000002a.

**$sp** is the CPU's stack pointer register. It holds the address of the top of the used stack. The four words are 4 bytes each, so they sit at ...de60, ...de64, ...de68 and ...de6c. The last word, 0x0000002a, is at 0x7fffffffde6c. That is the address of **stack_local_var**. 0x2a is 42 in hex so this is the raw memory of the variable.

**print heap_ptr** //Shows the address stored in the pointer

**x/1wd <address>** //Shows 1 word of memory at that address, in decimal
i.e **x/<count> <size> <format> <address>**

```
(gdb) print heap_ptr
$4 = (int *) 0x555555559010
(gdb) x/1wd heap_ptr
0x555555559010: 99
```

*Figure 11: Stack Variable and Stack Pointer Contents*

**print heap_ptr** gave 0x555555559010, the address returned by malloc(). It is inside the [heap] range.

**x/1wd heap_ptr** gave 99 at 0x555555559010, the value set with ***heap_ptr = 99.**

**How does x/1wd work?** The x command examines raw memory in the format

**1**: Count: show 1 unit of memory, **w**: Size: each unit is a word, which is 4 bytes i.e the size of an int

**d**: Format: show the value in decimal

So **x/1wd** heap_ptr tells GDB to go to the address stored in heap_ptr, read 4 bytes, and print them as a decimal number. That is why it printed 99. In the same way, **x/4wx $sp** means 4 words, shown in hex, starting at the stack pointer.

**continue** //Resumes the program until it finishes

**quit** //Leaves GDB

```
(gdb) continue
Continuing.
Ready for GDB inspection!
[Inferior 1 (process 513) exited normally]
(gdb) quit
wagleayden@Ayden:~/os-labs/LAB 4.2$
```

*Figure 12: Program Finished and GDB Closed*

# POST-LAB TECHNICAL ASSESSMENT

**Based on the addresses extracted during your lab runtime session, prove mathematically that the stack segment and the heap segment migrate toward one another during process expansion cycles.**

The heap starts at 0x555555559000 and grows upward, so its end address 0x55555557a000 increases when more memory is allocated. The stack ends at 0x7fffffff000 and grows downward, so its start address 0x7ffffffde000 decreases as more is used.

The gap between them is:

$0x7ffffffde000 \; - \; 0x55555557a000 \; = \; 0x2AAAAAA64000$

If the heap grows by $\Delta h$ and the stack by $\Delta s$. The new gap is:

$0x2AAAAAA64000 \; - \; \Delta h \; - \; \Delta s$, which is always smaller. The two move toward each other.

**If you run this compiled application simultaneously in two separate terminal windows, will both running programs display the exact same address locations or unique addresses? Explain your answer by discussing the concepts of Virtual Memory Spaces and operating system process isolation Same addresses or unique addresses in two terminals?**

Normally the addresses will be different because Linux randomizes the layout (ASLR) on each run. Inside GDB that randomization is off. That's why the addresses are the same each time.

Even when the printed addresses match, the two programs do not share memory. Each process has its own virtual address space and the OS maps it to different physical RAM. Therefore, one program cannot read or change the other's data.

# CONCLUSION

In this lab, a C program *demo.c* was written with one variable in each memory region: a global variable in .bss, a local variable on the stack, and a malloc() pointer in the heap. It was compiled with the -g flag and examined with GDB.

With info files, the .bss range was 0x555555558010 to 0x555555558018, and bss_global_var at 0x555555558014 was inside it. With readelf -l, .bss was in the RW LOAD segment and its file address 0x4014 matched the runtime address. Once the base address 0x555555554000 was added, because the program is a PIE. With info proc mappings, the heap was at 0x555555559000 to 0x55555557a000 and the stack was at 0x7fffffffde000 to 0x7fffffffff000. The stack variable (42) and the heap value (99) were read with print and both addresses fell inside their regions. The x/1wd command was explained as count 1, word size (4 bytes), decimal format.

The lab also showed that the heap grows upward and the stack grows downward, so they move toward each other. This lab helps one understand how a process's memory is arranged and how a debugger can inspect it.

# GITHUB REPOSITORY

GitHub Repository

# REFERENCES

Kerrisk, M. (2010). *The Linux programming interface*. No Starch Press.

Bryant, R. E., & O'Hallaron, D. R. (2015). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.